

A High-Fidelity Simulation Framework for Swarm Network-as-a-Service

Juan Esteban Beron Zapata, Balsam Alkouz, and Basem Shihada

Abstract—Swarm Network-as-a-Service (SNaaS) treats a coordinated unmanned aerial vehicle (UAV) swarm as reconfigurable airborne communication infrastructure. Validating such systems before deployment is hard, because the relevant behavior emerges only when flight dynamics, relay topology, radio feasibility, and fault recovery are coupled in a single runtime loop. Robotics simulators reproduce motion but treat communication as a passive output; wireless simulators reproduce channels but omit embodied swarm execution. We close this gap with a high-fidelity SNaaS framework in which link feasibility acts as an execution gate rather than a post-processing label: a packet advances only when its current hop satisfies range, SNR, and capacity constraints. The framework integrates Isaac Sim for embodied simulation, ROS 2 for coordination, PX4 and Pegasus for autopilot-accurate control, Sionna for telecom-aware link assessment, and a Mission Control plane for reproducible operation. We formalize the swarm as a time-varying relay graph, define availability, outage, and recovery as integrals over the feasibility gate, and prove that the recomputation policy is failure-monotone and restores service whenever a feasible path exists. We further establish a coverage-preservation condition that bounds the service region for which overlapping relay coverage is guaranteed. The result is a rigorous, reproducible instrument for quantitative study of service continuity, outage duration, recovery time, and communication-aware swarm reconfiguration.

Index Terms—Swarm Network-as-a-Service, SNaaS, UAV-assisted networks, non-terrestrial networks, high-fidelity simulation, digital twin, ROS 2, Isaac Sim, Sionna, PX4, Pegasus, fault tolerance, airborne communication infrastructure.

I. INTRODUCTION

Unmanned aerial vehicles (UAVs) are increasingly studied as agile non-terrestrial platforms for degraded or infrastructure-sparse settings [1], [2]. They are attractive when terrestrial communication is absent, overloaded, damaged, or unsafe to access. In disaster recovery, responders need connectivity even after ground infrastructure is partially destroyed. In remote or infrastructure-sparse regions, sensing and monitoring missions require connectivity even when terrestrial networks are unavailable. In temporary high-density events, fixed infrastructure may be insufficient for sudden demand surges. These scenarios motivate the use of UAVs not only as sensing or delivery vehicles, but also as airborne communication infrastructure that extends coverage, relays traffic, and restores service continuity on demand [3].

Juan Esteban Beron Zapata is with Universidad Pontificia Bolivariana, Medellin, Colombia, and the Networking Research Lab, KAUST, Thuwal, Saudi Arabia (e-mail: juan.beron@upb.edu.co).

Balsam Alkouz and Basem Shihada are with the Networking Research Lab, CEMSE Division, King Abdullah University of Science and Technology (KAUST), Thuwal, Saudi Arabia (e-mail: balsam.alkouz@kaust.edu.sa; basem.shihada@kaust.edu.sa).

The problem becomes more significant when UAVs are deployed as swarms. A single UAV may extend coverage locally, but a swarm can distribute load, provide multi-hop relay paths, enlarge the service region, and adapt the communication topology when nodes fail or radio conditions degrade [4], [5]. This motivates the Swarm Network-as-a-Service (SNaaS) paradigm, where drone-to-device and drone-to-drone interactions are modeled as composable connectivity services governed by service-level and quality-of-service requirements. SNaaS inherits the service-oriented view of Drone-as-a-Service (DaaS), but shifts the service objective from delivery, sensing, or inspection toward communication provisioning. Under this view, the swarm is not merely a collection of vehicles; it is a reconfigurable cyber-physical communication substrate.

Despite the importance of UAV-assisted communication, experimental validation remains fragmented. Communication-oriented studies frequently simplify UAVs as controllable aerial points and focus on coverage, placement, association, or capacity [4], [6]. Robotics-oriented simulators provide realistic motion, environments, and autonomy workflows, but often treat communication state as a secondary variable or omit link feasibility from the runtime loop [7]. Delivery-oriented DaaS simulators provide valuable examples of dynamic service simulation, but their operational primitives are packages, skyway nodes, landing pads, and delivery requests rather than relay topology, link metrics, outage behavior, and communication continuity. *The resulting gap is clear: SNaaS requires a simulation framework where embodied UAV behavior and communication service state are coupled during execution.*

Physical experimentation alone cannot close this gap. Multi-UAV field trials require airspace permissions, safety procedures, hardware availability, communication payloads, repeated operator effort, and favorable environmental conditions [8]. These constraints are prohibitive during early-stage design. They are especially severe when the researcher must inject failures, change topology, stress coverage, alter antenna assumptions, sweep communication ranges, or compare relay strategies under repeatable conditions. A high-fidelity simulator is therefore not a convenience layer; it is the scientific instrument that makes systematic SNaaS research feasible before real-world deployment [9].

The framework presented here is designed to serve that role. It provides a modular environment in which researchers can instantiate UAV swarms, configure service regions, evaluate communication-aware relay behavior, inject faults, inspect runtime topology, visualize coverage and packet progress, and export evidence for later analysis. The framework is

motivated by degraded urban environments and infrastructure-less remote regions, but it is not limited to those examples. A preliminary conference demonstration introduced the core relay-recovery workflow [10]; this article generalizes it into a complete simulation framework with a formal runtime model. It supports arbitrary scenario definitions and is intended as a reusable experimental foundation for UAV-assisted networking, non-terrestrial networks, communication-aware autonomy, and service-oriented swarm control.

We summarize our main contributions as follows:

- We motivate a dedicated SNaaS simulator by showing that the target system is simultaneously a robotic swarm, a communication service, and an operator-controlled experiment.
- We derive a requirement-driven toolchain in which Isaac Sim, ROS 2, PX4/Pegasus, Sionna, and Mission Control each satisfy one distinct, non-substitutable requirement.
- We design a runtime model that makes relay graphs, topology modes, branch-level parallelism, link feasibility, outage states, and dashboard-to-scene synchronization first-class objects.
- We formalize an analytical model in which communication feasibility gates packet advancement, and we prove that the topology-recomputation policy is failure-monotone, service-restoring, and coverage-preserving under explicit conditions.
- We position the framework as a research instrument for quantitative studies of service availability, outage duration, recovery time, and communication-aware swarm reconfiguration.

II. RELATED WORK

This section reviews six bodies of work that the framework builds on and departs from. We move from service-oriented UAV abstractions, through delivery-oriented and robotics simulators, to UAV-assisted communication, non-terrestrial networks, and telecom-aware digital twins. We then consolidate the gap and position the proposed framework against representative prior art in Table I.

A. Drone-as-a-Service and Service-Oriented UAV Systems

The service-oriented interpretation of UAV systems provides the conceptual foundation for this work. The DaaS survey in [11] frames drones as service providers and organizes the field through functions, research tasks, and application domains. It introduces a layered view in which drone capabilities, computing resources, and user-facing services are separated but jointly orchestrated. This distinction is essential for SNaaS simulation. The framework cannot be reduced to raw flight dynamics, because the target system is a service ecosystem in which UAV mobility, computation, communication, and mission requirements interact.

DaaS also identifies communication-as-a-service and drone-swarm-as-a-service as important research directions [11]. This is directly aligned with the present framework. The UAVs under study are not merely vehicles that execute trajectories; they are communication-bearing entities whose value is measured

through coverage, link quality, relay feasibility, and service continuity. The DaaS perspective therefore prevents the framework from collapsing into a narrow visual demonstration and instead motivates a modular research platform with explicit service roles.

Swarm-based drone services have also been studied in the delivery domain. SDaaS formulations treat a swarm as a service that transports packages across skyway segments under payload, battery, and charging constraints [12], [13]. This literature is highly relevant, because it shows how swarms can be modeled as composable service units. However, the service target differs fundamentally. In SDaaS, the functional service is package delivery; in SNaaS, the functional service is connectivity. This difference changes the simulator requirements. Delivery simulators prioritize payload transfer, skyway traversal, node charging, and delivery time, whereas SNaaS simulators must prioritize relay topology, communication feasibility, outage behavior, and service continuity.

B. Swarm Network-as-a-Service

SNaaS extends the service paradigm to UAV-assisted connectivity. The SNaaS model in [14] defines drone-to-device and drone-to-drone interactions as atomic services and composes them into end-to-end connectivity services under QoS and SLA requirements. It further distinguishes direct, clustered, and parallel composition strategies and studies how queueing-aware composition can adapt to load and latency constraints. This formalization is central to the present article. It defines what the simulator is expected to host experimentally: a swarm connectivity service rather than a collection of independent UAVs.

However, the SNaaS composition model and the framework proposed here address different layers of the research problem. The former establishes the service abstraction and composition logic. The latter provides an embodied experimental substrate in which topology, mobility, link feasibility, fault events, runtime reconfiguration, and evidence collection can be observed under controlled conditions. The framework is therefore complementary to SNaaS theory. It supplies the digital testing environment required to explore how SNaaS behaves when communication service logic is embedded in a simulated cyber-physical swarm.

C. Dynamic Simulation Frameworks for Drone Services

The strongest structural inspiration is the dynamic and immersive framework for drone delivery services in skyway networks proposed by Lin *et al.* [9]. That work shows that a simulation framework can be a scientific contribution in its own right. It motivates simulation as a response to the cost, safety, and regulatory difficulty of physical drone experiments. It also organizes the system into a simulator core, external control logic, real-time communication protocols, 3D visualization, and automated data export. The architecture is valuable because it separates user-defined scheduling logic from the simulator core and treats data collection as part of the research instrument.

The present work follows the same framework-oriented philosophy but targets a different service domain. The framework in [9] is tailored to service-based drone delivery in skyway networks. It models nodes, edges, landing pads, payloads, charging behavior, and delivery requests. In contrast, the proposed framework targets SNaaS, where the primary experimental variables are relay topology, link feasibility, communication range, antenna assumptions, SNR, capacity, outage state, and fault recovery. The proposed system inherits the methodological rigor of dynamic drone-service simulation while replacing delivery semantics with communication-service semantics.

Another relevant line of work studies failure-sentient swarm-based drone delivery services [15]. That work predicts soft failures, evaluates failure severity, and adapts service composition so that degraded drones do not disrupt package delivery. It is important here because it establishes that swarm-service systems must model failures proactively rather than as rare exceptions. Nevertheless, it remains delivery-oriented. Its objective is delivery success and delivery time under soft failure. The present framework adapts the broader failure-aware philosophy to SNaaS, where failures affect relay paths, link feasibility, branch availability, packet progression, and communication continuity.

D. Robotics and UAV Simulation Platforms

General robotics and UAV simulators provide the building blocks for high-fidelity experimentation. Gazebo has long been used for robot and UAV simulation with ROS integration [16]. AirSim provides high-fidelity visual and physical simulation for autonomous vehicles, including UAV workflows [17]. SwarmLab supports drone-swarm algorithm development and evaluation in MATLAB [18]. MAV-swarms explores swarm stabilization along a path using onboard relative localization [19]. Broader surveys of aerial swarm robotics catalog the coordination, control, and perception primitives these platforms expose [20]. PX4 multi-vehicle SITL, bridged into ROS 2 through Micro XRCE-DDS, has become a standard substrate for autopilot-accurate multi-UAV experiments [21].

Existing robotics simulators do not directly solve the SNaaS experimentation problem. They generally do not provide the communication-service semantics required by an airborne network-as-a-service platform. In particular, they do not natively maintain relay-service topology, gate packet advancement on link feasibility, expose Sionna-aware SNR and capacity as runtime service variables, or represent branch-level service continuity as part of the simulation loop. A recent survey of network-simulation tools for UAV communication confirms this fragmentation. Most tools emphasize either mobility realism or channel realism, but rarely both inside one runtime loop [7]. These platforms are therefore valuable components or baselines, but not complete SNaaS research instruments.

E. UAV-Assisted Communication and Non-Terrestrial Networks

The communications literature establishes the networking importance of UAVs. Multi-UAV placement and user-association work shows that UAV-assisted communication is

not only a coverage problem; 3D placement, mobility, user association, and link quality must be considered jointly [6]. Joint placement-and-routing studies extend this view to multi-hop aerial relaying, where intermittent links and frequent topology changes couple routing with relocation [4]. Surveys of UAV cellular communications identify propagation, interference, latency, energy, regulation, and security as persistent deployment challenges [1], [2]. These works justify aerial communication platforms, but they often abstract UAVs as controllable nodes rather than embodied agents in a runtime swarm-service workflow.

SNaaS is also a non-terrestrial-network (NTN) problem. Surveys on the evolution of NTN from 5G to 6G position UAVs, alongside HAPS and satellites, as airborne segments of an integrated space-air-ground fabric [22], [23]. Outlook articles on the connected sky argue that UAV segments must satisfy concrete link, mobility, and reliability constraints rather than idealized coverage assumptions [3]. A high-fidelity SNaaS simulator gives these NTN abstractions an embodied, link-aware testbed. Recent wireless research further shows that contextual information improves communication decisions. Multimodal visual/RF user association and beam-forming demonstrates the value of combining environmental awareness with radio metrics [24]. The camel-grazing monitoring system in [25] illustrates a practical remote scenario in which UAV-based sensing is constrained by limited terrestrial communication and electrical infrastructure. These examples reinforce the same principle: communication-aware autonomy requires tools that fuse physical context, mission state, and radio-state feedback.

F. Telecom-Aware Digital Twins and Wireless Simulation

Recent tools connect high-fidelity simulation with wireless modeling. AirTwinX couples Isaac Sim, Pegasus/PX4, and Sionna ray tracing for advanced air mobility digital twins [26]. Ns3Sionna integrates realistic Sionna channels into ns-3 network simulation [27]. Sionna itself provides a modern open-source foundation for physical-layer and ray-tracing-oriented wireless research [28], [29]. These works are important because they show that wireless realism benefits from geometry-aware and context-aware simulation.

The proposed framework differs in its service semantics and runtime objective. AirTwinX targets advanced air mobility digital-twin modeling, while Ns3Sionna strengthens network simulation by importing realistic channels. The present framework targets SNaaS execution. It maintains relay topology, disseminates per-UAV state, evaluates link feasibility, gates packet movement, injects failures, recomputes service paths, and records service continuity. It therefore adds a ROS 2-centered swarm-service layer that turns 3D simulation and link evaluation into an inspectable SNaaS workflow.

G. Research Gap and Positioning

The prior literature reveals a clear gap. DaaS and SNaaS provide service abstractions but not a high-fidelity embodied simulator. Delivery-oriented drone-service simulators provide excellent framework inspiration but target package movement

rather than connectivity. Robotics simulators provide physical and visual realism but lack communication-service semantics. Wireless simulators provide communication rigor but often omit embodied swarm execution. Telecom-aware digital twins improve radio realism but do not necessarily provide SNaaS-specific runtime semantics, such as relay-topology maintenance, packet gating, branch-level parallel relay behavior, and service-recovery instrumentation. Table I summarizes this positioning and makes explicit how the proposed framework differs from representative prior work.

III. REQUIREMENTS AND JUSTIFICATION

A simulation framework earns the label *high-fidelity* only if its design decisions are traceable to the phenomenon it must reproduce. This section develops that traceability for SNaaS. We begin from the nature of the system under study, derive the requirements it imposes, and show that those requirements determine the toolchain and the runtime capabilities almost uniquely. The argument is deliberately presented as a single chain of reasoning rather than a catalogue of features, so that each component of the framework can be justified by the requirement it satisfies and by the alternative it rules out.

The starting observation is that a SNaaS deployment is not one system but three superimposed systems. It is a robotic swarm, whose behavior is governed by flight dynamics, embodiment, and spatial geometry. It is a communication service, whose value is measured through relay feasibility, link quality, and service continuity. And it is an operator-controlled experiment, whose scientific worth depends on repeatability, controlled disturbance, and a preserved evidence trace. These three views are not separable post hoc. A relay path that is feasible on a map may be infeasible once embodiment and distance are imposed; a swarm motion that looks correct in a robotics viewer may silently violate communication continuity; and a result that cannot be reproduced under a fixed configuration is not evidence at all. The central methodological claim of this work is therefore that the three views must be coupled inside a single runtime loop, because the coupling is precisely where the interesting failures live.

This claim explains why physical experimentation, although indispensable for final validation, cannot carry early-stage SNaaS research. Multi-UAV field trials require airspace permissions, safety procedures, hardware, communication payloads, repeated operator effort, and favorable conditions [8]. None of these scales to the disturbance-control regime that the science demands: comparing relay topologies, injecting node failures, sweeping communication range, varying antenna assumptions, and repeating each configuration until the variance is understood. The same constraints that make field trials valuable for confirmation make them prohibitive for exploration. A simulator is thus not a convenience that approximates the field; it is the only instrument in which the coupled, disturbance-rich experiments can be run at all [9]. This mirrors the trajectory of mature drone-service research, where dynamic simulation became a first-class contribution precisely because it unlocked experiments that hardware could not support.

From this reasoning the requirements follow directly, each tied to one of the three superimposed systems. Embodied UAV realism and swarm coordination serve the robotic view. Communication awareness and service semantics serve the service view, and they carry a sharp sub-requirement: link feasibility must influence execution, not merely annotate it. Runtime reconfiguration, evidence generation, and extensibility serve the experimental view. The decisive requirement, the one that separates a SNaaS simulator from every adjacent tool, is that these requirements hold *simultaneously and continuously* during a run. A tool that satisfies any proper subset is useful as a component but incomplete as an instrument: a network-only simulator overestimates feasibility by ignoring embodiment, a robotics-only simulator renders convincing motion while measuring no continuity, and a delivery-oriented service framework models swarms as services yet lacks relay and link semantics.

Because no single off-the-shelf tool meets the full requirement set, the design problem becomes one of principled composition rather than selection. Table II makes the trade-off explicit across representative candidate stacks. To keep the comparison honest, we score each candidate on the five requirement axes and express the overall suitability as a transparent weighted sum,

$$S_{\text{overall}} = w_e S_e + w_u S_u + w_m S_m + w_t S_t + w_r S_r, \quad \sum_k w_k = 1, \quad (1)$$

where S_e, S_u, S_m, S_t, S_r denote environment fidelity, UAV realism, middleware integration, telecom awareness, and runtime extensibility. Equation (1) is a design discipline, not a universal benchmark: it forces every weight, and therefore every selection, to be defended against the requirements rather than against habit. Under any weighting that respects the decisive telecom sub-requirement, the Isaac Sim + Pegasus + ROS 2 + Sionna composition dominates, because it is the only candidate that is simultaneously high on embodiment and high on in-loop link evaluation.

The selected components are therefore not interchangeable parts but complementary answers to distinct requirements. Isaac Sim provides the high-fidelity world and visualization substrate [30]. Pegasus specializes that world for multirotor UAVs [31], and PX4 supplies autopilot-accurate control semantics with a path to sim-to-real validation through software-in-the-loop and a DDS bridge [21], [32]. ROS 2 is the coordination backbone that makes swarm state inspectable rather than hidden inside an opaque loop [33]. Sionna contributes the communication-aware layer that turns geometry into link feasibility, SNR, and capacity [28], [29], [34]. The coherence of the composition is best seen by subtraction: Isaac Sim without Sionna is visually rich but communication-blind; Sionna without Isaac Sim is communication-aware but disembodied; ROS 2 alone is a message bus, not a simulator; and PX4 with Pegasus but no telecom layer is UAV-centered but not SNaaS-centered. The contribution is the integration itself, governed by a ROS 2 coordination layer that the adjacent literature does not provide.

That same integration is what endows the framework with the runtime capabilities that distinguish it in practice, and

TABLE I
POSITIONING OF THE PROPOSED FRAMEWORK AGAINST REPRESENTATIVE PRIOR WORK.

Work / Tool	Service	Embodied	Telecom runtime	Fault/outage	Main distinction
DaaS survey [11]	High	Low	Conceptual	Conceptual	Provides service taxonomy and layered foundations, not a runtime simulator.
SNaaS model [14]	High	Low	Analytical	Limited	Defines connectivity-service composition, not embodied experimentation.
Dynamic delivery framework [9]	High	High	Low	Delivery-specific	Targets delivery, skyways, payloads, and energy, not relay communication services.
Failure-sentient SDaaS [15]	High	Medium	Low	High	Studies delivery failure prediction, not SNaaS link feasibility and outage behavior.
Multi-hop placement/routing [4]	Medium	Low	Analytical	Network-level	Optimizes placement and routing analytically, without embodied runtime gating.
AirTwinX [26]	Medium	High	High	Medium	Targets AAM digital twins rather than SNaaS service execution.
Ns3Sionna [27]	Medium	Low	High	Network-level	Improves network realism without embodied UAV swarm orchestration.
Proposed framework	High	High	High	High	Couples UAV embodiment, relay topology, link feasibility, runtime faults, and evidence collection.

TABLE II
REQUIREMENT-DRIVEN COMPARISON OF CANDIDATE STACKS FOR SNAAS EXPERIMENTATION. THE DECISIVE COLUMN IS **TELECOM**, SINCE ONLY STACKS THAT EVALUATE LINK FEASIBILITY INSIDE THE LOOP CAN GATE EXECUTION.

Stack	World	UAV	ROS/MW	Telecom	Runtime	Fit	Rationale
Gazebo + ROS 2	M	H	H	L	M	M	Strong robotics baseline, but communication modeling must be added externally.
Gazebo + ROS 2 + Sionna	M	H	H	H	H	H	Strong balanced baseline; lower visual fidelity than Isaac-centered stack.
Isaac Sim + ROS 2	H	M	H	L	M	M-H	Excellent world layer, but communication-blind without a telecom engine.
Isaac Sim + Pegasus + ROS 2	H	H	H	L	H	H	Strong UAV realism; still lacks integrated link evaluation.
Isaac Sim + Pegasus + ROS 2 + Sionna	H	H	H	H	H	H+	Best fit: high-fidelity embodiment, UAV specialization, middleware, and telecom awareness.
CARLA + ROS 2 + Sionna	H	L-M	M	H	M	M	Strong urban context, but less natural for multirotor swarm control.
AirSim + ROS 2 + Sionna	M-H	M	M	H	M	M	Useful UAV baseline, weaker long-term modularity for SNaaS experimentation.

H: high, M: medium, L: low. Scores are project-specific design ratings, not universal benchmarks.

these capabilities are best understood as the requirements made operational rather than as a separate feature list. Because the swarm is modeled as a communication-service substrate, relay paths, branch assignments, topology modes, and service continuity are first-class runtime objects rather than visual decorations. Because link evaluation lives inside the loop, feasibility gates packet advancement during execution, which is the single behavior that most sharply separates a SNaaS simulator from a visual swarm demo. Because the service view admits more than one composition strategy, the framework represents chain, parallel, and forest topologies as genuinely distinct processes: parallel mode is a set of independent branch streams, each with its own packet cursor and outage state, not one chain drawn across several rows. Because the experimental view demands control, the operator can reconfigure topology, swarm size, range, thresholds, motion, antennas, and fault state during a run; and because it demands evidence, every status snapshot, link metric, event, and visual artifact is persisted for later quantitative analysis. These capabilities are exercised and formalized in Sections V and VI, where the gated loop and its recovery guarantees are made precise.

IV. SYSTEM ARCHITECTURE

A. Architectural Overview

The framework follows a client–core architecture. The client side provides operator-facing control, monitoring, and visual access. The containerized computing core executes coordination, communication analysis, and embodied simulation. Figure 1 illustrates this structure and the closed loop that

couples runtime control, telecom evaluation, embodied visualization, and evidence collection. The decisive property is that communication feasibility is computed inside the loop, so it constrains execution rather than merely annotating it afterward.

B. Client and Mission-Control Layer

The client layer contains the interfaces used by the researcher. It includes scenario commands, runtime configuration, failure injection, metrics output, logs, WebRTC-based visualization, and the Mission Control dashboard. This layer is essential because the simulator is intended for interactive research, not only batch execution. A researcher must be able to configure a mission, change topology, introduce faults, inspect service state, and retrieve evidence without manually editing low-level simulator internals.

C. ROS 2 Coordination and Middleware Layer

The ROS 2 coordination layer is the central integration backbone. It maintains runtime topics, relay and topology nodes, configuration updates, failure events, link requests, link metrics, and dashboard streams. In the current simulator, this layer maintains the semantic state of the swarm: which drones are active, which are failed, which branch each drone belongs to, which relay mode is active, which packets are advancing, and which links are feasible. States are not hidden inside an opaque simulator loop; they are represented through topics, events, and structured messages, which makes the system recordable, debuggable, replayable, and scientifically inspectable.

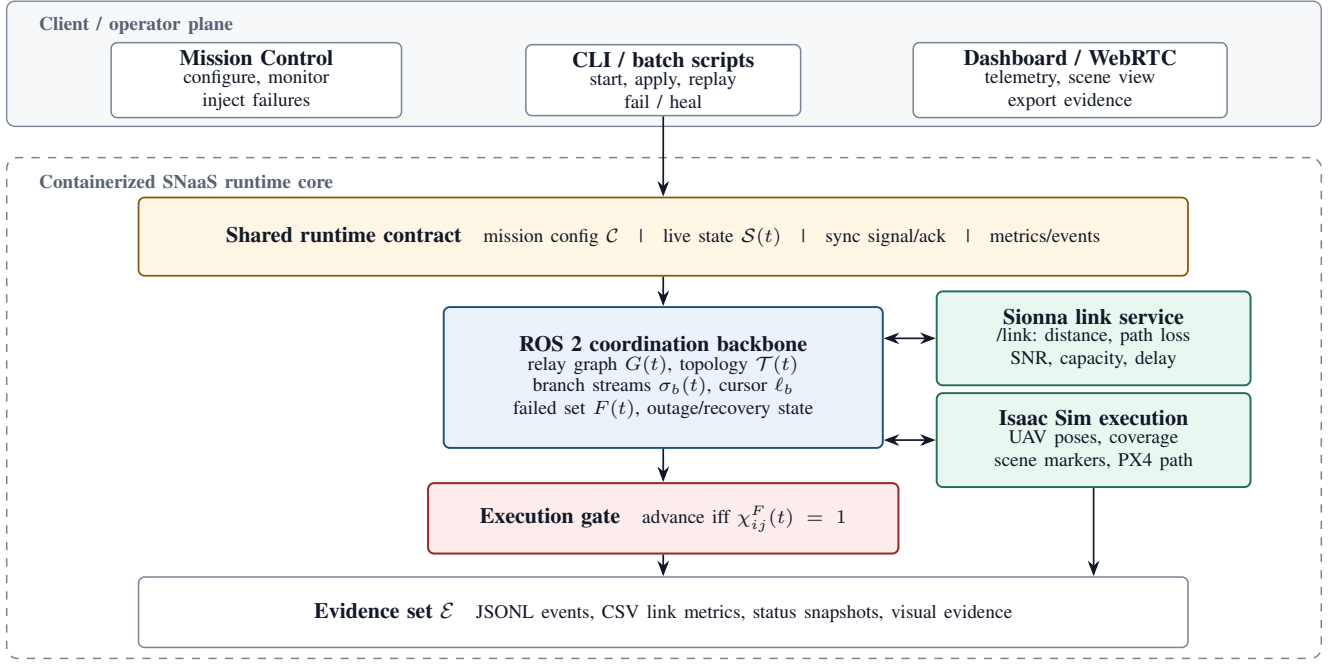


Fig. 1. Architecture of the SNaaS simulator. The operator plane configures and observes missions, while the containerized runtime core couples a shared runtime contract, ROS 2 coordination, Sionna-based link evaluation, Isaac Sim embodied execution, and evidence persistence. The execution gate is the key architectural invariant: relay advancement, outage handling, recovery, and visual packet motion are conditioned on the same failure-aware feasibility test $\chi_{ij}^F(t)$.

D. Sionna Link Service

The Sionna link service is the communication-aware component of the architecture. It receives link requests containing transmitter and receiver positions, antenna assumptions, and relevant physical parameters. It returns link metrics such as distance, path loss, received power, SNR, propagation delay, and capacity. These outputs are consumed by the coordination layer to decide whether relay communication can proceed. This design choice makes communication feasibility an execution constraint: packets advance only when the corresponding link satisfies the configured feasibility rules, which is the main difference between a SNaaS simulator and a purely visual swarm demo.

E. Isaac Sim Visualization and Embodied Execution Layer

The Isaac Sim layer provides the 3D environment, UAV models, motion visualization, status markers, antennas, coverage indicators, and visual evidence. It receives state and topology packets from the coordination layer and updates the scene accordingly. It also returns state feedback that the coordination layer can use. In this sense, Isaac Sim is both a visualization layer and an embodied execution environment. The visualization layer is scientifically relevant because SNaaS behavior is spatial: coverage, distance, relay feasibility, and link range all depend on positions and environment geometry, so visual evidence verifies that the dashboard topology corresponds to the embodied scene.

F. Mapping Conceptual Layers to Modules

The DaaS/SNaaS conceptual layers map naturally onto concrete modules. The drone layer is realized through Isaac

Sim, Pegasus, PX4, UAV models, motion state, and visual markers. The computing and control layer is realized through ROS 2 nodes, topology services, mission logic, the Sionna interface, monitoring, and logging. The user and mission layer is realized through Mission Control, CLI commands, scenario configuration, failure injection, WebRTC visualization, and evidence retrieval. This mapping shows that the architecture is the implementation of the service-oriented model that motivates SNaaS, not an arbitrary collection of tools.

V. OPERATIONAL MISSION WORKFLOW

The architecture in Section IV describes *where* computation happens; this section formalizes *when* and *in what order* it happens during a mission. A SNaaS mission is a structured lifecycle rather than a free-running animation. Figure 2 captures this lifecycle as a control flow with an inner per-hop loop and explicit disturbance handling. The workflow is the operational contract between the operator, the coordination layer, the link service, and the embodied scene.

A. Operator-Facing Workflow Interfaces

The conceptual workflow in Fig. 2 is exposed to the researcher through a Mission Control plane rather than through simulator internals. This interface layer is part of the experimental apparatus: it materializes the mission configuration, supervises the relay-service state, injects disturbances, synchronizes the Isaac Sim scene, and preserves the evidence stream used for post-run analysis. Its role is therefore different from a conventional visualization dashboard. Mission Control is the operator-facing view of the runtime contract between the

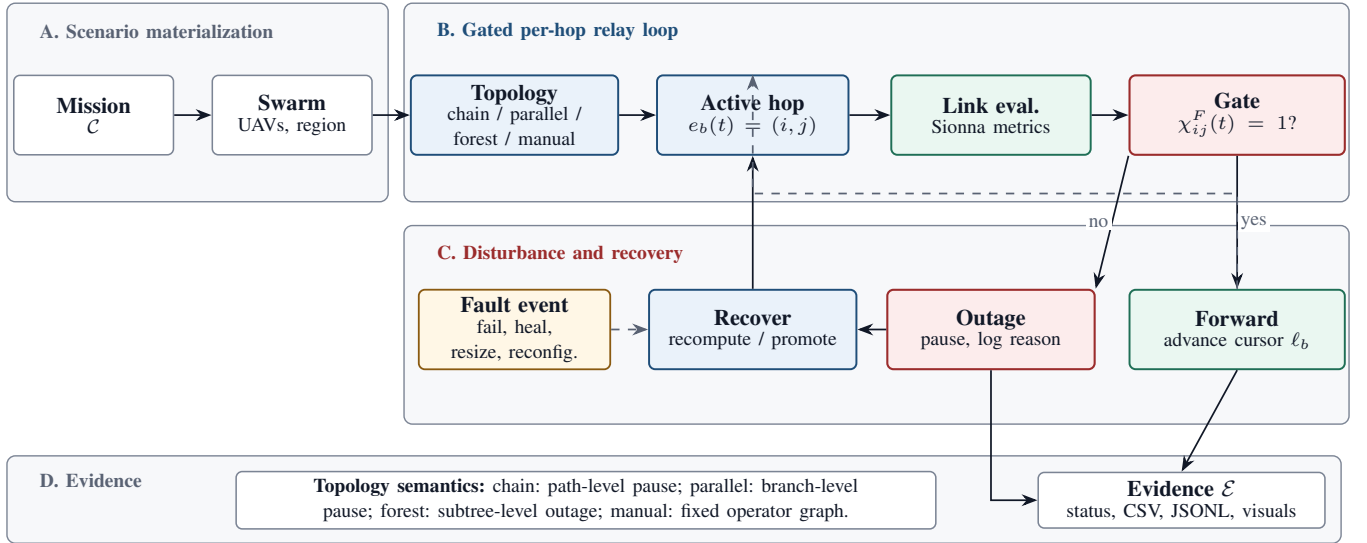


Fig. 2. Operational workflow of one SNaaS simulation run. Phase A materializes the mission configuration into an embodied swarm and relay topology. Phase B executes the gated per-hop relay loop: the active hop is evaluated by the link service and can advance only if $\chi_{ij}^F(t) = 1$. Phase C handles disturbances by recording outages, recomputing topology, and promoting standby UAVs only when they restore a feasible path. Phase D preserves the run as structured evidence for replay and quantitative analysis.

configuration object $\mathcal{C}$, the live state $\mathcal{S}(t)$, the relay topology $\mathcal{T}(t)$, the link-feasibility service, and the evidence set $\mathcal{E}$.

Figure 3 shows the main Overview panel, which serves as the operational state-of-truth during a run. The upper status region reports whether the containerized runtime, configuration snapshot, and Isaac synchronization path are active. The mission-state card records the experiment identifier, transmission mode, topology, active relay count, failed set, standby reserve, packet/sample state, availability estimate, and synchronization acknowledgments. The one-click mission controls write the shared configuration, trigger synchronization, start or restore the Docker/Isaac stack, and force visual snapshots. Below these controls, the fleet counters summarize the configured drones, active relays, failed nodes, standby reserves, availability, and outage events. The digital-twin map renders the current relay graph and service-region boundary, while the Link/SLA panel reports the active hop, distance, SNR, capacity, failed-endpoint state, and other link-service outputs. Finally, the execution-gate readiness panel lists the endpoint, range, SNR, capacity, and failure constraints that must hold before a packet, branch, or visual relay marker is advanced. This makes the central SNaaS invariant inspectable during execution: relay progress is permitted only when the active hop satisfies $\chi_{ij}^F(t) = 1$.

The remaining surfaces in Fig. 4 expose the workflow at different levels of abstraction. The Mission Designer belongs to Phase A because it defines the controlled variables of the experiment: relay count, standby reserve, service-region dimensions, communication radius, hard-outage distance, altitude envelope, motion pattern, wind/turbulence parameters, carrier frequency, bandwidth, transmit power, SNR threshold, capacity threshold, target availability, and traffic rate. Writing this configuration produces the reproducible mission object consumed by the runtime and the Isaac synchronization signal. Live Telemetry belongs to Phases B and D: it displays

the link-metric stream generated by the per-hop feasibility loop, including SNR, capacity, distance-to-range margin, link-state distribution, top loaded hops, outage/failure events, and raw metric records. The Algorithm Lab is the research extension point. It lists validated Python strategies and their entry points, such as `plan_positions(context)`, `on_failure(context, idx)`, and `select_standby(context)`, allowing researchers to test custom control, placement, recovery, or telecom-aware policies without modifying the core ROS 2 relay controller. The Isaac/WebRTC view provides embodied evidence: it shows the selected world, UAV models, visual relay markers, service geometry, and the spatial consistency between the dashboard topology and the scene state.

World loading and recovery are treated as workflow events rather than manual visualization steps. The World/Environment surface and Isaac loader select the scene asset or digital-twin world used to instantiate the service region. Once loaded, the world becomes part of the evidence context because link distance, coverage overlap, occlusion assumptions, and visual relay state are interpreted relative to that scene. During a disturbance, a fault or operator action updates the failed set $F(t)$, invalidates links incident to failed UAVs, and triggers topology recomputation. A standby UAV is promoted only if the recomputed path satisfies the same feasibility gate used during nominal forwarding. The Isaac views in Fig. 5 show how this behavior is preserved visually: the urban world anchors the service context, the reconfiguration view records post-failure relay organization, and the ranging view exposes the geometric service budget that constrains forwarding.

Table III summarizes the interface surfaces and their workflow roles. The table is useful because each surface corresponds to a different scientific function: configuration, execution, perturbation, synchronization, extension, or evidence.

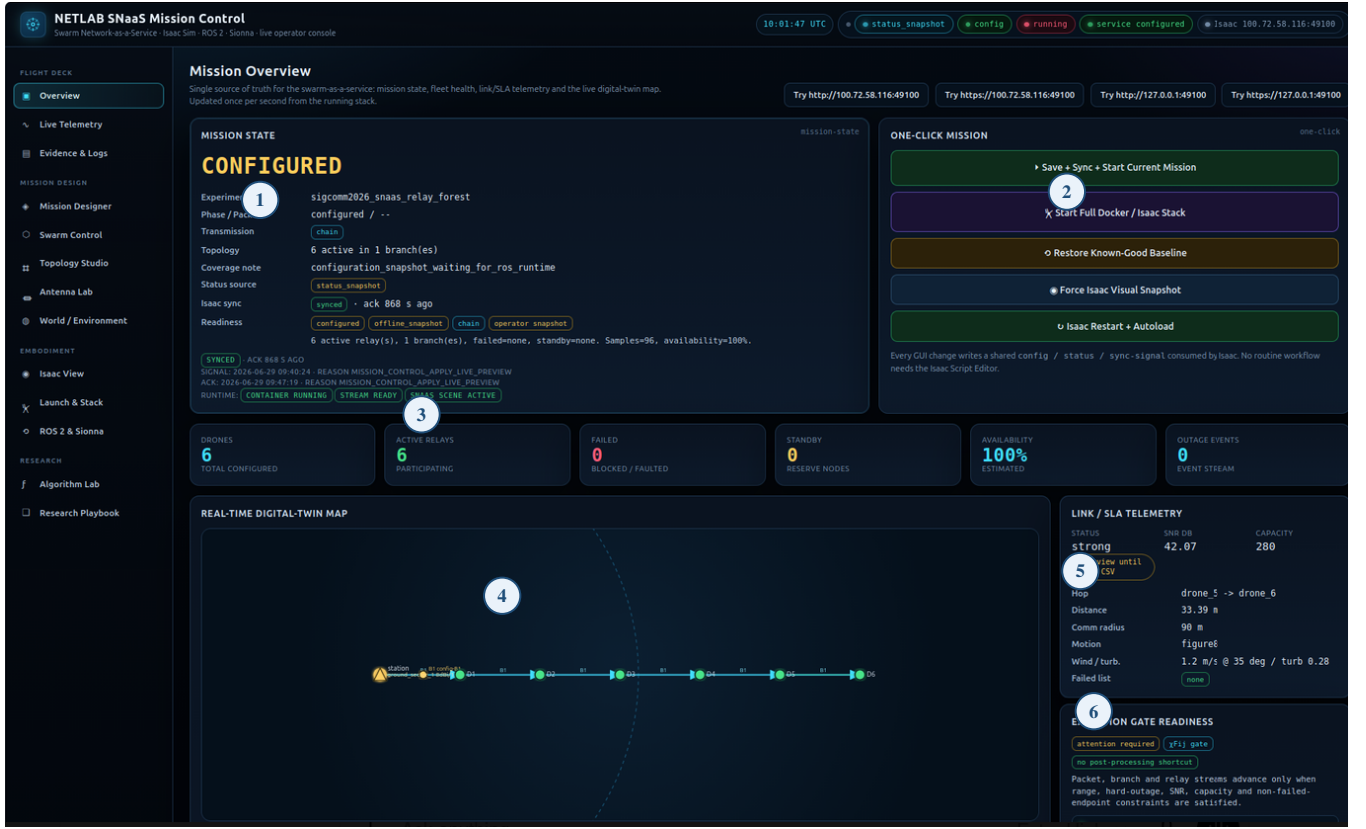


Fig. 3. Mission Control Overview used as the runtime state-of-truth for a SNaas mission. The numbered callouts identify: (1) mission state, status source, and Isaac synchronization; (2) one-click lifecycle actions that write the shared configuration and start or restore the stack; (3) fleet-health counters for configured, active, failed, and standby UAVs; (4) the real-time digital-twin relay map; (5) active-hop Link/SLA telemetry returned by the communication layer; and (6) execution-gate readiness, where relay advancement is conditioned on the failure-aware feasibility gate $\chi_{ij}^F(t)$.

This separation keeps the simulator reproducible while still allowing interactive operation.

B. Phase A: Scenario Configuration

The mission begins when the operator submits a configuration $\mathcal{C}$ through Mission Control. The framework instantiates the swarm, places the ground station, defines the service region, and constructs the initial relay graph $G(t)$ and active topology $\mathcal{T}(t)$. This phase fixes the controlled variables of the experiment: swarm size, branch count, topology mode, communication range, antenna assumptions, and motion pattern. Because $\mathcal{C}$ is persisted, the run is reproducible and the parameter trace is part of the evidence.

C. Phase B: Gated Per-Hop Relay Loop

The runtime then enters the gated relay loop, which is the operational heart of the framework. For each active hop $e_b(t) = (i, j)$, the coordination layer queries Sionna with the current transmitter and receiver poses and receives link metrics. The failure-aware feasibility gate $\chi_{ij}^F(t)$ then decides the outcome. If the gate is satisfied, the packet advances to the next hop. If it is not, the branch pauses and an outage event is recorded with its limiting cause. This loop encodes the central design principle: feasibility is an execution gate, not a post-hoc label.

D. Phase C: Disturbance and Recovery

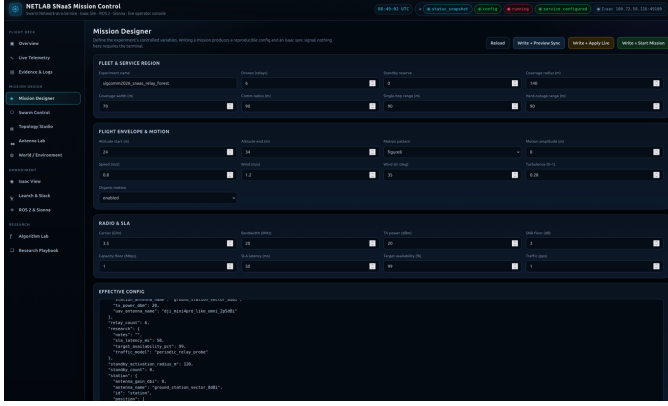
Disturbances enter the workflow as explicit operator actions or scheduled events. When a fault marks a UAV as failed, the failed set $F(t)$ is updated, the topology-recomputation policy $\mathcal{R}_{\text{topo}}$ runs, and standby UAVs are promoted only if they restore a feasible path. The recomputed topology re-enters the gated loop, so the effect of recovery is observed through the same feasibility test that governs nominal operation. This uniform treatment is what allows recovery time and outage duration to be measured consistently, and it is the behavior whose correctness we establish formally in Section VI.

E. Phase D: Evidence Persistence

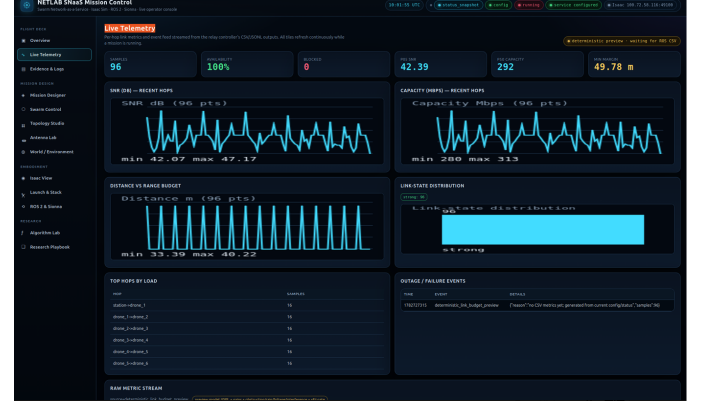
Every terminal event in the workflow persists into the evidence set $\mathcal{E}$. Advanced packets, paused branches, injected faults, topology changes, and link metrics are written to structured logs, CSV link-metric streams, and JSONL event traces. This phase closes the loop between runtime behavior and quantitative analysis, because the same artifacts that drive the dashboard also feed post-run evaluation of availability, outage, and recovery.

VI. ANALYTICAL BACKGROUND AND RUNTIME MODEL

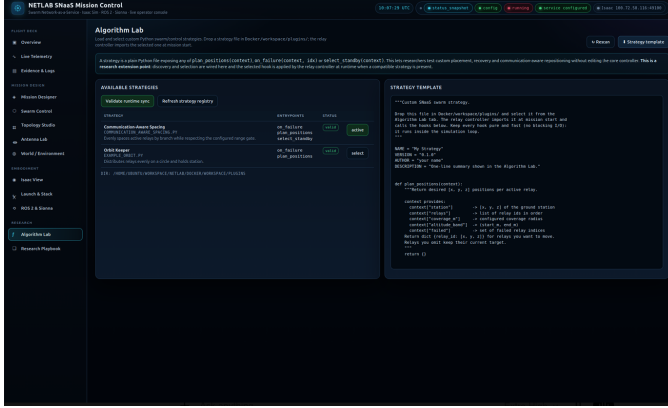
This section develops the formal model that underlies the workflow of Section V. We define the runtime objects, the



(a) Mission Designer: reproducible scenario variables and effective configuration.



(b) Live Telemetry: per-hop SNR, capacity, distance margin, link-state distribution, and outage events.

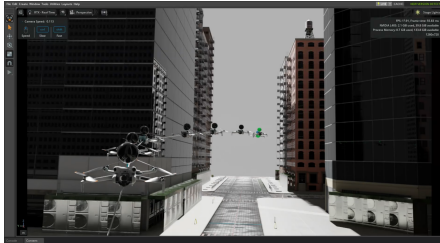


(c) Algorithm Lab: plugin-based extension point for custom placement, control, and recovery strategies.



(d) Isaac/WebRTC view: embodied UAV relay scene and visual synchronization with the runtime state.

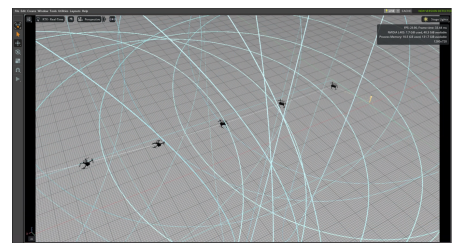
Fig. 4. Mission Control surfaces that implement the operational workflow. The Mission Designer fixes the experimental variables; Live Telemetry exposes the link-feasibility and outage evidence stream; the Algorithm Lab supports researcher-defined control or telecom strategies; and the Isaac/WebRTC view verifies that the relay topology and status markers are materialized in the embodied scene.



(a) Urban-world materialization: the selected environment anchors the embodied SNaas scenario.



(b) Fault/reconfiguration evidence: relay status markers and links after disturbance handling.



(c) Range-ring evidence: geometric service budget and hop feasibility become visually inspectable.

Fig. 5. Embodied evidence generated by Isaac Sim. These views complement the dashboard by showing where the service is materialized: the selected world defines the physical context, relay markers show runtime status and reconfiguration, and coverage or range rings expose the geometric constraints that feed the link-feasibility gate.

link model, and the feasibility gate, then state and prove the structural guarantees of the recovery policy and the coverage-preservation condition. The formalism makes the simulator's behavior auditable and connects every runtime artifact to a measurable quantity.

A. Configuration, State, and Evidence Objects

The implementation is organized around three runtime object families. The mission configuration $\mathcal{C}$ contains the number

of drones N , the number of branches q , the topology mode μ , the maximum single-hop range $d_{\max}$, the hard-outage range d_{hard} , altitude bounds, coverage radius, motion pattern, antennas, worlds, and fault settings. The live state $\mathcal{S}(t)$ contains drone positions, branch streams, active relays, failed nodes, standby nodes, packet state, link metrics, and event traces. The evidence set $\mathcal{E}$ includes status snapshots, event logs, link-metric streams, visualization state, and synchronization

TABLE III
MISSION CONTROL SURFACES AND THEIR ROLE IN THE SNAAS OPERATIONAL WORKFLOW.

Interface surface	Workflow role	Main elements exposed	Scientific function
Mission Designer	Phase A: scenario materialization	Relay count, standby reserve, service-region dimensions, communication range, hard-outage distance, altitude, motion, wind, carrier, bandwidth, SNR/capacity floors, target availability, effective JSON configuration	Defines the reproducible mission object $\mathcal{C}$ and fixes the controlled variables used for later comparison.
Overview	Phases A–D: operational state-of-truth	Mission state, one-click lifecycle actions, Isaac sync, fleet counters, digital-twin relay map, active-hop SLA telemetry, execution-gate readiness	Connects configuration, runtime state, link feasibility, visual synchronization, and evidence in one inspectable surface.
Live Telemetry	Phases B and D: metric stream and evidence	SNR, capacity, distance-to-range margin, link-state distribution, top hops by load, outage/failure events, raw metrics	Verifies that packet advancement and outages are tied to measured link-feasibility variables rather than post-run labels.
Swarm Control / Topology Studio	Phases B and C: topology operation	Topology mode, relay membership, branch state, fail/heal controls, active/failed/standby labels	Allows controlled topology changes and fault injection while preserving a visible relation to $G(t)$, $F(t)$, and $\mathcal{T}(t)$.
Algorithm Lab	Phases A and C: research extension	Strategy registry, validation status, selected strategy, plugin entry points for planning, failure handling, and standby selection	Enables researchers to inject custom control, placement, recovery, or telecom-aware policies without editing the core simulator.
World / Environment and Isaac View	Phases A–D: embodied execution and visual evidence	Selected scene/world, UAV poses, relay markers, coverage/range indicators, WebRTC stream, sync snapshots	Shows that the service graph is embodied in a spatial scene and that recovery or outage evidence is consistent with the digital-twin state.
Evidence & Logs	Phase D: persistence and replay	JSONL events, CSV link metrics, status snapshots, sync acknowledgments, visual screenshots	Converts runtime behavior into auditable artifacts for availability, outage-duration, recovery-time, and replay analysis.

acknowledgments. A run is the controlled transformation

$$\mathcal{R} : (\mathcal{C}, \mathcal{U}, \mathcal{W}) \mapsto \{\mathcal{S}(t), \mathcal{E}\}_{t \in [0, T]}, \quad (2)$$

where $\mathcal{U}$ denotes operator actions and $\mathcal{W}$ denotes the world and environmental assumptions.

B. Relay Graph

The swarm is a time-varying directed graph

$$G(t) = (V(t), E(t)), \quad (3)$$

where $V(t)$ contains the ground station, active UAV relays, standby UAVs, antennas, and optionally mobile users, and $E(t)$ contains candidate communication links. With $\mathbf{p}_i(t) \in \mathbb{R}^3$ the position of node i , the inter-node distance is

$$d_{ij}(t) = \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|_2. \quad (4)$$

The active set is $V_a(t) = V(t) \setminus F(t)$, where $F(t)$ is the set of failed UAVs. A link can be geometrically short yet invalid if one endpoint is failed.

C. Communication Link Service

The link service returns physical-layer metrics for a transmitter–receiver pair. A free-space reference model is

$$L_{\text{fsp},ij}(t) = 20 \log_{10} \left(\frac{4\pi f_c d_{ij}(t)}{c} \right), \quad (5)$$

with f_c the carrier frequency and c the speed of light. The implementation-level path loss adds environment, obstruction, and margin terms,

$$L_{ij}(t) = L_{\text{fsp},ij}(t) + \kappa_{\text{env}}(t) + \kappa_{\text{obs}}(t) + \kappa_{\text{margin}}. \quad (6)$$

The received power, noise floor, SNR, capacity, and delay are

$$P_{r,ij}(t) = P_t + G_t + G_r - L_{ij}(t), \quad (7)$$

$$N_B = -174 + 10 \log_{10}(B) + NF, \quad (8)$$

$$\gamma_{ij}^{\text{dB}}(t) = P_{r,ij}(t) - N_B, \quad (9)$$

$$C_{ij}(t) = B \log_2 \left(1 + 10^{\gamma_{ij}^{\text{dB}}(t)/10} \right), \quad (10)$$

$$\tau_{ij}(t) = \frac{d_{ij}(t)}{c}. \quad (11)$$

These match the structure of the current link-service implementation, which reports distance, path loss, received power, SNR, capacity, delay, and a qualitative state.

D. Link Feasibility Gate

Definition 1 (Hop feasibility). A hop (i, j) is feasible at time t if and only if

$$\chi_{ij}(t) = \begin{cases} 1, & d_{ij}(t) \leq d_{\text{max}}, d_{ij}(t) < d_{\text{hard}}, \\ & \gamma_{ij}(t) \geq \gamma_{\text{min}}, C_{ij}(t) \geq C_{\text{min}}, \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

The simulator uses $\chi_{ij}(t)$ as a runtime gate: if $\chi_{ij}(t) = 0$, the corresponding packet or branch pauses and an outage event is recorded. The feasibility margin

$$m_{ij}(t) = \min\{d_{\text{max}} - d_{ij}(t), \gamma_{ij}(t) - \gamma_{\text{min}}, C_{ij}(t) - C_{\text{min}}\} \quad (13)$$

identifies the limiting cause of outage: a non-positive margin pinpoints whether range, SNR, or capacity is binding.

E. Topology Modes and Branch Streams

The framework supports chain, parallel, forest, and manual topology modes. In chain mode, the active relay sequence is $\mathcal{P}_{\text{chain}} = (s, d_1, \dots, d_M)$ and its feasibility is

$$\chi_{\mathcal{P}}(t) = \prod_{(i,j) \in \mathcal{P}_{\text{chain}}} \chi_{ij}(t). \quad (14)$$

In parallel mode, the swarm decomposes into independent branches $\mathcal{B} = \{B_1, \dots, B_q\}$ with $B_b = (s, d_{b,1}, \dots, d_{b,m_b})$. Each branch carries an independent packet stream

$$\sigma_b(t) = (\text{id}_b(t), \ell_b(t), \phi_b(t), \pi_b(t), r_b(t)), \quad (15)$$

where id_b is the packet identifier, ℓ_b the hop index, ϕ_b the phase, $\pi_b \in \{0, 1\}$ the pause indicator, and r_b the pause reason. Branch feasibility is $\chi_{B_b}(t) = \prod_{(i,j) \in B_b} \chi_{ij}(t)$. Parallel transmission is therefore a set of branch-level relay processes, each with its own cursor and outage behavior, not a single chain drawn across rows.

F. Failure-Aware Feasibility and Recovery Guarantees

Let $F(t) \subseteq V(t)$ be the failed UAVs. A link is invalid if either endpoint is failed:

$$\chi_{ij}^F(t) = \chi_{ij}(t) \cdot \mathbf{1}\{i \notin F(t)\} \cdot \mathbf{1}\{j \notin F(t)\}. \quad (16)$$

The packet-advancement rule follows directly: $\ell_b(t + \Delta t) = \ell_b(t) + 1$ if $\chi_{e_b(t)}^F(t) = 1$, and $\ell_b(t + \Delta t) = \ell_b(t)$ otherwise, with the pause reason recorded among $\{\text{range}, \text{snr}, \text{capacity}, \text{failed node}\}$. Topology reconfiguration is the operator

$$\mathcal{T}(t^+) = \mathcal{R}_{\text{topo}}(\mathcal{T}(t^-), F(t), V_a(t), \mathcal{C}), \quad (17)$$

which may remove failed nodes, promote standby nodes, rebalance branches, or preserve a manually fixed topology. We now establish the two structural properties that make the recovery behavior trustworthy.

Definition 2 (Service feasibility). A topology $\mathcal{T}$ is *service-feasible* at time t if it contains at least one source-to-sink path $\mathcal{P} \subseteq \mathcal{T}$ all of whose hops satisfy $\chi_{ij}^F(t) = 1$. We write $\Phi(\mathcal{T}, t) = 1$ in that case and $\Phi(\mathcal{T}, t) = 0$ otherwise.

Theorem 1 (Failure-monotone service restoration). *Let $\mathcal{R}_{\text{topo}}$ select, among all source-to-sink paths over the active set $V_a(t)$, any path that maximizes the minimum hop margin $m_{ij}(t)$, and let it promote a standby node only when doing so yields $\Phi = 1$. Then, for any failure event at time t_f :*

- 1) (**Monotonicity**) *failing an additional node never increases the set of feasible paths: if $F(t_2) \supseteq F(t_1)$ and the geometry is fixed, then $\Phi(\mathcal{T}, t_2) \leq \Phi(\mathcal{T}, t_1)$ for every fixed $\mathcal{T}$;*
- 2) (**Restoration**) *if at least one service-feasible path exists over $V_a(t_f^+)$, then $\mathcal{R}_{\text{topo}}$ returns a topology with $\Phi = 1$;*
- 3) (**Soundness**) *if no service-feasible path exists over $V_a(t_f^+)$, then $\mathcal{R}_{\text{topo}}$ returns $\Phi = 0$ and the branch enters a recorded outage rather than forwarding on an infeasible hop.*

Proof. Monotonicity. By (16), $\chi_{ij}^F(t)$ contains the factors $\mathbf{1}\{i \notin F(t)\} \mathbf{1}\{j \notin F(t)\}$, which are non-increasing in F . Any path feasible under $F(t_2)$ uses only nodes in $V \setminus F(t_2) \subseteq V \setminus F(t_1)$ and links with $\chi^F = 1$ under the larger failed set, hence feasible under $F(t_1)$. Thus the feasible-path set under $F(t_2)$ is a subset of that under $F(t_1)$, giving $\Phi(\mathcal{T}, t_2) \leq \Phi(\mathcal{T}, t_1)$.

Restoration. Suppose a service-feasible path exists over $V_a(t_f^+)$. The set of source-to-sink paths over a finite V_a is finite, so the maximization of the minimum hop margin is

well defined and attains its maximum on some path $\mathcal{P}^*$. By Definition 1, a hop has $\chi_{ij}^F = 1$ iff $m_{ij} \geq 0$. Because at least one feasible path exists, its minimum margin is non-negative, so the maximizer $\mathcal{P}^*$ also has non-negative minimum margin, i.e. every hop of $\mathcal{P}^*$ satisfies $\chi_{ij}^F = 1$. Hence $\Phi = 1$. Standby promotion only enlarges V_a , which cannot reduce the feasible-path set, so the guarantee is preserved.

Soundness. If no service-feasible path exists, the maximizer over the empty feasible set is vacuous, and the advancement rule with gate (16) forbids forwarding on any hop with $\chi_{ij}^F = 0$. The branch therefore pauses and the outage is recorded, giving $\Phi = 0$. $\square$

Remark 1. Theorem 1 formalizes the recovery behavior exercised in the demonstration: failed relays are removed, the active path is recomputed over the survivors, and standby drones are promoted only when they restore a feasible path. It guarantees the simulator never advertises forwarding it cannot deliver.

G. Coverage Bubbles and a Service-Region Bound

For a drone i with coverage radius r_i , the coverage indicator over a service region Ω is $\psi_i(x, t) = \mathbf{1}\{\|x - \mathbf{p}_i(t)\|_2 \leq r_i\}$, the coverage multiplicity is $\Psi(x, t) = \sum_{i \in V_a(t)} \psi_i(x, t)$, and the coverage and redundancy ratios are

$$\eta_\Omega(t) = \frac{1}{|\Omega|} \int_\Omega \mathbf{1}\{\Psi \geq 1\} dx, \quad \eta_\Omega^{(r)}(t) = \frac{1}{|\Omega|} \int_\Omega \mathbf{1}\{\Psi \geq r\} dx. \quad (18)$$

The next result makes precise when a linear relay chain preserves overlapping coverage along the service region, which is the condition violated by the overextended region in the demonstration.

Proposition 1 (Overlapping-coverage condition). *Consider M relays placed along a corridor of length D with common coverage radius r and common single-hop budget $d_{\max} \geq 2r$. If the relays are spaced by $\delta = D/(M - 1)$ and*

$$\delta \leq 2r \quad \text{and} \quad \delta \leq d_{\max}, \quad (19)$$

then the union of coverage disks contains the corridor centerline with no gap, the chain is hop-feasible on geometry, and $\eta_\Omega^{(2)} > 0$ on every interior overlap. Conversely, if $\delta > 2r$, a coverage gap of length $\delta - 2r$ appears between consecutive relays and the chain loses contiguous coverage.

Proof. Place relay k at arclength $(k-1)\delta$ along the centerline. Consecutive disks of radius r centered at distance δ overlap iff $\delta \leq 2r$; the overlap region has positive length $2r - \delta \geq 0$, so $\Psi \geq 2$ there and $\eta_\Omega^{(2)} > 0$. Under $\delta \leq 2r$ the disks tile the centerline without gaps, giving $\eta_\Omega = 1$ along it. Hop feasibility on geometry requires $\delta \leq d_{\max}$, which holds by assumption. If instead $\delta > 2r$, the open interval between the two disk boundaries has length $\delta - 2r > 0$ and lies outside every disk, so $\Psi = 0$ there and contiguous coverage is lost. $\square$

Remark 2. Proposition 1 explains the distance-induced outage qualitatively reported in [10]: holding r and M fixed while stretching D increases δ ; once δ exceeds $2r$ (or $d_{\max}$), overlap

is lost and forwarding pauses. The condition gives a closed-form design bound for the service-region length that a swarm of given size and range can cover.

H. Topology Modes as Distinct Service Processes

The composition strategy is not a cosmetic choice; it determines how feasibility, load, and failure propagate through the service. We therefore treat each supported mode as a distinct stochastic process over the relay graph and make its packet semantics explicit. Figure 6 renders the three regimes side by side, with the live packet shown on the link it currently occupies.

In *chain* mode, a single packet stream traverses one source-to-sink path $\mathcal{P}_{\text{chain}} = (s, d_1, \dots, d_M)$, so the path is feasible only if every hop is feasible:

$$\chi_{\mathcal{P}}(t) = \prod_{(i,j) \in \mathcal{P}_{\text{chain}}} \chi_{ij}(t). \quad (20)$$

The chain maximizes reach per UAV but is series-reliable: a single infeasible hop suspends the whole stream, a property also observed in analytical multi-hop UAV relay reliability studies [35].

In *parallel* mode, the swarm is partitioned into q branches $\mathcal{B} = \{B_1, \dots, B_q\}$ that share the station s but carry independent packet streams. Each branch $B_b = (s, d_{b,1}, \dots, d_{b,m_b})$ owns a state tuple

$$\sigma_b(t) = (\text{id}_b(t), \ell_b(t), \phi_b(t), \pi_b(t), r_b(t)), \quad (21)$$

with packet identifier, hop cursor, phase, pause indicator, and pause reason, and its feasibility is $\chi_{B_b}(t) = \prod_{(i,j) \in B_b} \chi_{ij}(t)$. The crucial semantic distinction is that parallel mode is *not* one chain drawn across several rows: the branches advance, stall, and recover independently, so a single branch outage degrades capacity without halting the service.

In *forest* mode, branches may themselves fork, forming a directed tree rooted at s in which interior relays fan out to several children. Let $\text{ch}(i)$ denote the children of relay i . A subtree rooted at i is feasible when the hop into i is feasible and at least one child subtree remains feasible, giving the recursive condition

$$\Theta_i(t) = \chi_{\text{pa}(i),i}^F(t) \cdot \bigvee_{j \in \text{ch}(i)} \Theta_j(t), \quad (22)$$

with $\Theta_\ell(t) = 1$ at any leaf ℓ that reaches an intended sink. Forest mode trades raw reach for branch-level redundancy: a failed interior relay prunes only its subtree, and sibling subtrees continue to serve. These three feasibility laws, series in (20), parallel-independent for χ_{B_b} , and tree-recursive in (22), are the formal reason the framework represents the modes as separate processes rather than as one parameterized path.

I. Failure-Aware Recovery and Coverage Geometry

The topology laws above describe nominal operation; the framework's defining behavior is what happens under disturbance. Figure 7 visualizes Theorem 1 and Proposition 1 on a representative chain. Panel (a) shows the nominal regime:

relays spaced by $\delta \leq 2r$ produce overlapping coverage disks and a contiguous active path. Panel (b) shows the post-failure regime: two relays fail, the recomputation policy removes them, a standby node is promoted, and a feasible source-to-sink path is restored over the survivors. The coverage union is re-tiled so that contiguous coverage is preserved wherever a feasible path exists, exactly as quantified by (19). The two panels share the geometry assumed by Proposition 1, so the figure is a faithful illustration of the analytical claim rather than a separate schematic.

J. Availability, Outage, and Recovery Metrics

For a horizon $[0, T]$, branch availability, swarm availability, branch outage, and recovery time are

$$A_b = \frac{1}{T} \int_0^T \chi_{B_b}^F(t) dt, \quad A_{\text{swarm}} = \frac{1}{q} \sum_{b=1}^q A_b, \quad (23)$$

$$O_b = \int_0^T (1 - \chi_{B_b}^F(t)) dt, \quad T_{\text{rec}} = t_r - t_f, \quad (24)$$

where t_f is the failure time and t_r the restoration time. By Theorem 1, t_r is finite whenever a feasible path exists over the survivors, so T_{rec} is well defined and directly measurable from the event log.

K. Isaac Synchronization Model

The implementation synchronizes the operator interface, ROS 2 runtime, and Isaac Sim via configuration and status snapshots. With λ_s the timestamp of the latest synchronization signal and λ_a the latest acknowledgment, the visual lag is $\Delta_{\text{sync}} = \lambda_s - \lambda_a$, and the scene is synchronized when $\Delta_{\text{sync}} \leq \epsilon_{\text{sync}}$. This guards the validity of visual evidence: the dashboard and the embodied scene must agree before a screenshot is interpreted as ground truth.

L. Runtime Object Summary

Table IV links the analytical model to the main runtime objects.

VII. RESERVED SECTIONS FOR METHODOLOGY AND EVALUATION

The sections below define the remaining structure of the full article.

A. Simulation Methodology

This section captures the concrete implementation of the simulator, including container deployment, ROS 2 topic structure, Isaac Sim scene generation, Sionna link-service calls, PX4/Pegasus integration, status synchronization, and Mission Control workflows.

B. Experimental Scenarios

This section defines representative scenarios such as degraded urban infrastructure, infrastructure-less remote regions, dense temporary events, and remote sensing missions. These scenarios are examples rather than restrictions on the framework.

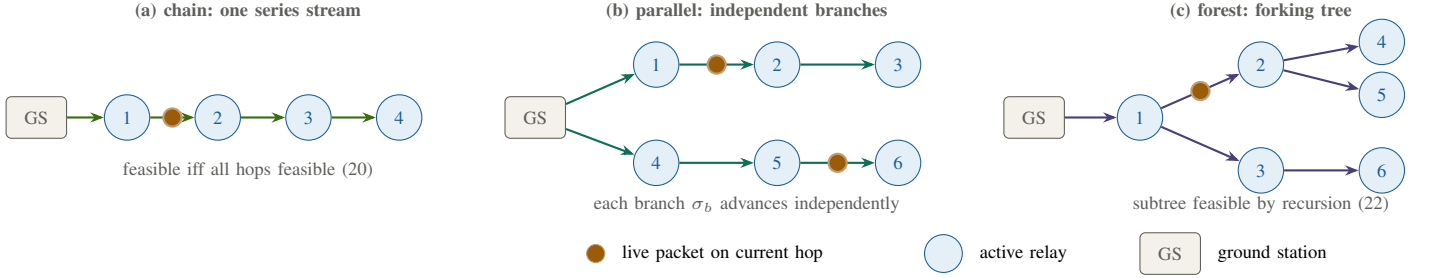


Fig. 6. The three supported relay-topology modes, shown as distinct service processes with the live packet on its current hop. (a) Chain: a single series-reliable stream; one infeasible hop suspends the whole path (20). (b) Parallel: q independent branch streams sharing the station, each with its own packet cursor and outage state, so capacity degrades gracefully under a single-branch outage. (c) Forest: branches fork into a tree, giving subtree-level redundancy through the recursive feasibility law (22).

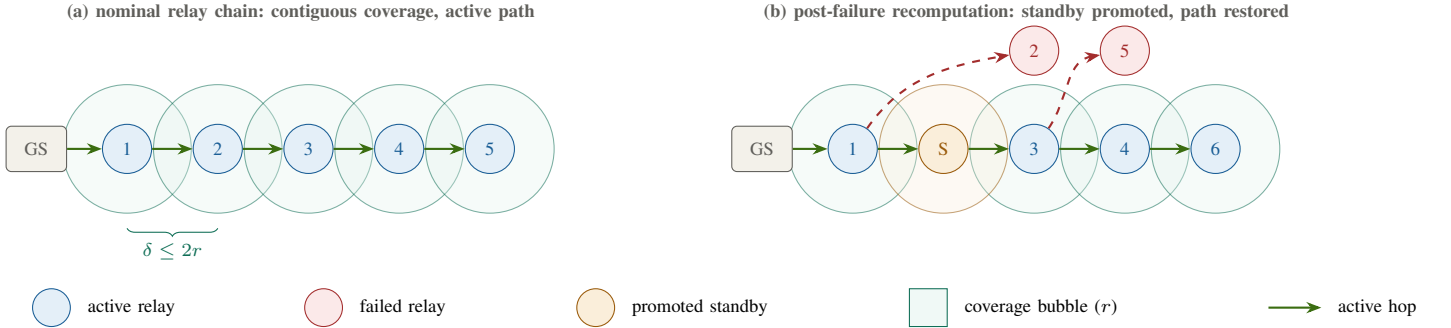


Fig. 7. Swarm relay-graph reconfiguration and coverage. (a) Nominal chain: relays are spaced by $\delta \leq 2r$, so coverage disks overlap and the active path is contiguous (Proposition 1). (b) Post-failure: relays 2 and 5 fail; the recomputation policy removes them, promotes a standby node (S), and restores a feasible source-to-sink path over the survivors (Theorem 1). Coverage is re-tiled so that overlapping coverage is preserved along the restored path.

TABLE IV
RUNTIME OBJECTS AND THEIR ANALYTICAL ROLES IN THE SNAAS SIMULATOR.

Runtime object	Information carried	Analytical role	Evidence use
Mission configuration	drones, branches, ranges, antennas, worlds, motion, topology mode	Defines $\mathcal{C}$ and topology constraints	Reproducibility and parameter trace
Latest status snapshot	active relays, failed nodes, branch streams, packet phase, link status	Defines $\mathcal{S}(t)$	Dashboard state and post-run analysis
Sionna link response	distance, path loss, received power, SNR, capacity, delay	Defines $\chi_{ij}(t)$ and margins	Link metrics, outage labels, SLA proxies
Branch stream state	packet id, hop cursor, phase, pause reason	Defines $\sigma_b(t)$	Parallel-mode evaluation and branch health
Isaac sync signal/ack	timestamps, scene update reason, config hash	Defines Δ_{sync}	Visual-evidence validity
Event log	failures, heals, resets, topology changes, outages	Defines disturbance and recovery sequence	Recovery time and causal trace

C. Evaluation Metrics

This section formalizes the metrics used in the experimental campaign, including link feasibility ratio, packet advancement rate, branch availability, outage duration, recovery time, topology recomputation latency, SNR distribution, capacity distribution, sync latency, and service-continuity score.

D. Results and Validation

This section reports controlled simulation campaigns comparing chain, parallel, forest, and manual topologies under varying swarm size, communication range, mobility pattern, fault injection, and antenna configuration.

E. Discussion and Limitations

This section interprets the results and discusses limitations, including the current level of physical realism, simplified

energy assumptions, interference modeling, user mobility, heterogeneous UAVs, and the path toward sim-to-real validation.

F. Conclusion

This section synthesizes the completed article after the quantitative results and validation campaign are finalized.

ACKNOWLEDGMENT

This work is partially supported by the King Abdulah University of Science and Technology (KAUST) Global Fellowship Program, Award No. RFS-KGFP2024-6784. The statements made herein are solely the responsibility of the authors.

REFERENCES

- [1] A. Fotouhi, H. Qiang, M. Ding, M. Hassan, L. G. Giordano, A. Garcia-Rodriguez, and J. Yuan, "Survey on UAV Cellular Communications: Practical Aspects, Standardization Advancements, Regulation, and Security Challenges," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3417–3442, 2019.
- [2] X. Gu and G. Zhang, "A Survey on UAV-Assisted Wireless Communications: Recent Advances and Future Trends," *Computer Communications*, vol. 208, pp. 44–78, 2023.
- [3] M. Mozaffari, X. Lin, and S. Hayes, "Toward 6G with Connected Sky: UAVs and Beyond," *IEEE Communications Magazine*, vol. 59, no. 12, pp. 74–80, 2021.
- [4] A. Shamsoshoara, F. Afghah, and J. Ashdown, "Joint Mobility-Aware UAV Placement and Routing in Multi-Hop UAV Relaying Systems," in *Ad Hoc Networks*, Springer, 2021, pp. 55–69.
- [5] D. Clerigues, J. Wubben, C. T. Calafate, J.-C. Cano, and P. Manzoni, "Enabling Resilient UAV Swarms Through Multi-Hop Wireless Communications," *EURASIP Journal on Wireless Communications and Networking*, vol. 2024, Art. 39, 2024.
- [6] H. El Hammouti, M. Benjillali, B. Shihada, and M.-S. Alouini, "Learn-As-You-Fly: A Distributed Algorithm for Joint 3D Placement and User Association in Multi-UAVs Networks," *IEEE Transactions on Wireless Communications*, vol. 18, no. 12, pp. 5831–5844, 2019.
- [7] H. Jiang, Z. Liu, J. Liu, Y. Zhao, and X. Wang, "Network Simulation Tools for Unmanned Aerial Vehicle Communications: A Survey," *International Journal of Communication Systems*, vol. 37, no. 13, Art. e5878, 2024.
- [8] E. Petritoli, F. Leccese, and L. Ciani, "Reliability and Maintenance Analysis of Unmanned Aerial Vehicles," *Sensors*, vol. 18, no. 9, Art. 3171, 2018.
- [9] J. Lin, B. Alkous, A. Bouguettaya, and A. Abu Safia, "Dynamic and Immersive Framework for Drone Delivery Services in Skyway Networks," *ACM Transactions on Internet Technology*, vol. 25, no. 1, Art. 7, 2025.
- [10] J. E. Beron Zapata, B. Alkous, and B. Shihada, "DEMO: A High-Fidelity Simulator for Swarm Network-as-a-Service," *SIGCOMM Posters and Demos*, 2026.
- [11] A. Hamdi, B. Alkous, B. Shahzaad, A. Bouguettaya, A. G. Neiat, F. Salim, and D. Y. Kim, "Drone-as-a-Service: Research Challenges and Directions," *Proceedings of the IEEE*, vol. 113, no. 5, pp. 416–442, 2025.
- [12] B. Alkous, A. Bouguettaya, and S. Mistry, "Swarm-based Drone-as-a-Service (SDaaS) for Delivery," in *Proc. IEEE International Conference on Web Services (ICWS)*, pp. 441–448, 2020.
- [13] B. Alkous, A. Abusafia, A. Lakhdari, and A. Bouguettaya, "In-flight Energy-driven Composition of Drone Swarm Services," *IEEE Transactions on Services Computing*, 2022.
- [14] B. Alkous, O. Amin, and B. Shihada, "Swarm Network-as-a-Service (SNaaS)," in *Proc. IEEE International Conference on Web Services (ICWS)*, 2026, accepted.
- [15] B. Alkous, A. Bouguettaya, and A. Lakhdari, "Failure-Sentient Composition for Swarm-Based Drone Services," arXiv:2305.13892, 2023.
- [16] J. Meyer, A. Sendobry, S. Kohlbrecher, U. Klingauf, and O. von Stryk, "Comprehensive Simulation of Quadrotor UAVs Using ROS and Gazebo," in *Proc. International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAR)*, 2012.
- [17] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," arXiv:1705.05065, 2017.
- [18] E. Soria, F. Schiano, and D. Floreano, "SwarmLab: A MATLAB Drone Swarm Simulator," arXiv:2005.02769, 2020.
- [19] M. Saska, "MAV-Swarms: Unmanned Aerial Vehicles Stabilized Along a Given Path Using Onboard Relative Localization," in *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 894–903, 2015.
- [20] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen, and V. Kumar, "A Survey on Aerial Swarm Robotics," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 837–855, 2018.
- [21] PX4 Autopilot Project, "Multi-Vehicle Simulation with ROS 2," PX4 User and Developer Guide, online documentation, accessed 2026.
- [22] M. M. Azari, S. Solanki, S. Chatzinotas, O. Kodheli, H. Sallouha, A. Colpaert, J. F. Mendoza Montoya, S. Pollin, A. Haqiqatnejad, A. Mostaani, E. Lagunas, and B. Ottersten, "Evolution of Non-Terrestrial Networks From 5G to 6G: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2633–2672, 2022.
- [23] G. Geraci, D. Lopez-Perez, M. Benzaghta, and S. Chatzinotas, "Integrating Terrestrial and Non-Terrestrial Networks: 3D Opportunities and Challenges," *IEEE Communications Magazine*, vol. 61, no. 4, pp. 42–48, 2023.
- [24] Y. Li, Y. Liu, and W. Yu, "Multimodal Visual Image Based User Association and Beamforming Using Graph Neural Networks," *IEEE Xplore*, Article 11062505, 2025.
- [25] M. Chen, C. P. Lau, and B. Shihada, "Intelligent Aerial-Based Recognition and Positioning System for Camel Grazing," technical report, KAUST, 2025.
- [26] A. Dey, M. McManus, Z. Zhang, G. Sun, N. Mastronarde, E. S. Bentley, and Z. Guan, "AirTwinX: A High-Fidelity Digital Twin for Advanced Air Mobility with Ray Tracing," in *Proc. IEEE Consumer Communications & Networking Conference (CCNC)*, 2025.
- [27] A. Zubow, Y. Pilz, S. Rosler, and F. Dressler, "Ns3 Meets Sionna: Using Realistic Channels in Network Simulation," arXiv:2412.20524, 2024.
- [28] J. Hoydis, S. Cammerer, F. Ait Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An Open-Source Library for Next-Generation Physical Layer Research," arXiv:2203.11854, 2022.
- [29] J. Hoydis, F. Ait Aoudia, S. Cammerer, M. Nimier-David, N. Binder, G. Marcus, and A. Keller, "Sionna RT: Differentiable Ray Tracing for Radio Propagation Modeling," arXiv:2303.11103, 2023.
- [30] NVIDIA, "Isaac Sim Documentation," online documentation.
- [31] Pegasus Simulator Team, "Pegasus Simulator Documentation," online documentation.
- [32] PX4 Autopilot Project, "PX4 User Guide and ROS 2 Integration Documentation," online documentation.
- [33] Open Robotics, "ROS 2 Documentation," online documentation.
- [34] NVIDIA, "Sionna Documentation," online documentation.
- [35] N. Ning, S. Zhou, W. Bao, and X. Li, "A Study on the Maximum Reliability of Multi-UAV Cooperation Relay Systems," *Sensors*, vol. 24, no. 9, Art. 2886, 2024.